

CDE2310 User Manual

1. System Overview

The system is an Autonomous Mobile Robot (AMR) built on the TurtleBot3 Burger platform for warehouse intralogistics simulation. It autonomously explores an unknown environment, detects stations using fiducial markers, and delivers payloads (ping-pong balls) into receptacles.

The system integrates:

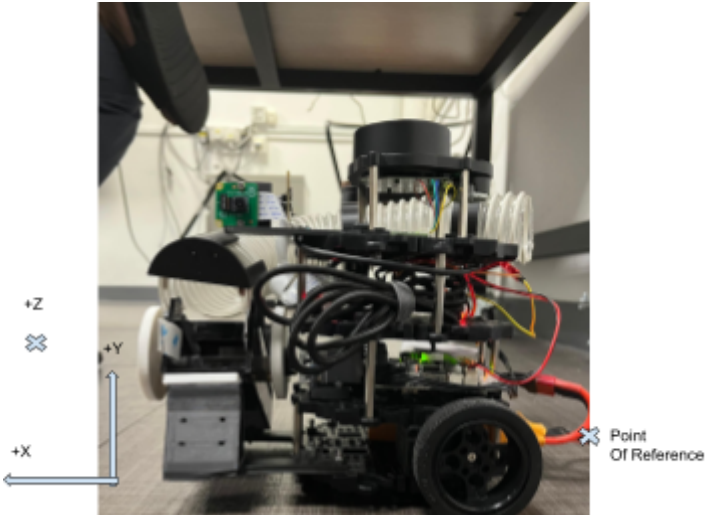
- **Navigation** (Nav2 + SLAM/exploration)
- **Perception** (ArUco docking + vision-based alignment)
- **Actuation** (servo-driven launcher)

Two delivery modes are implemented:

- **Station A:** deterministic timed firing
- **Station B:** event-triggered firing

2. Hardware Configuration

2.1 Mobile Base

Platform	TurtleBot3 Burger
Drive Type	Differential drive
Dimensions (L × W × H)	225mm x 180mm x 230mm
Weight	1469 g
Center of Gravity	x: 38.6, y: 31.1, z: 56.1 
Max linear velocity	0.22 m/s
Max rotational velocity	2.84 rad/s (162.72 deg/s)
Sensors	LDS-02 LiDAR, 3-axis Gyroscope, 3-Axis Accelerometer
Power Supply	LiPo battery (~11.1V, 1800 mAh)

Max Operating Time	1.5 - 2h
Charging time	2h

2.2 Compute & Control

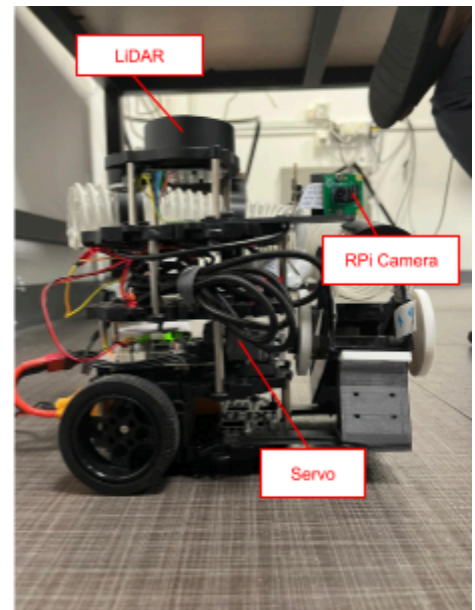
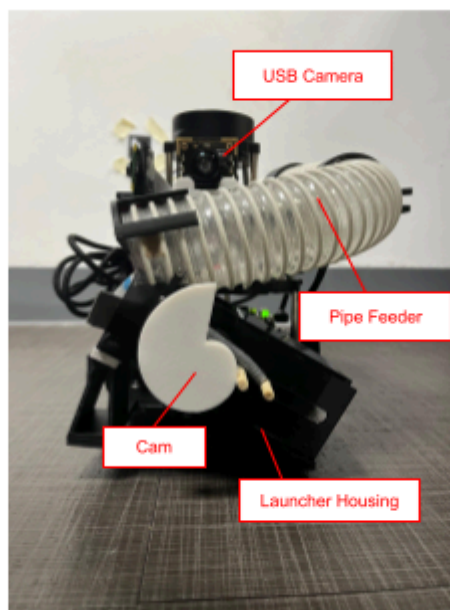
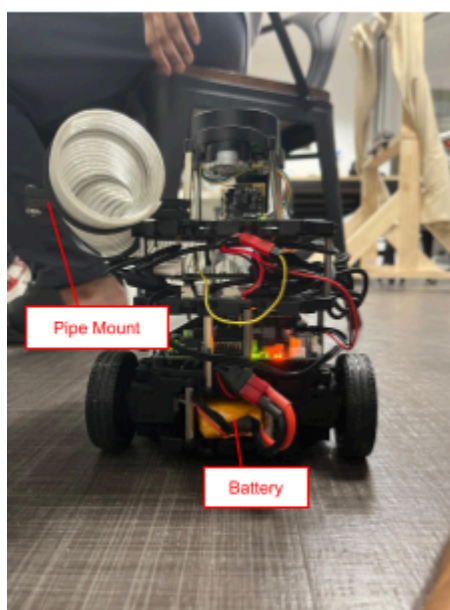
Main Computer	Raspberry Pi 4B
Motor Controller	OpenCR1.0
Communication	ROS 2 DDS

2.3 Perception Sensors

Front Camera (USB Camera)	• Purpose: ArUco marker detection & docking • Output: Station pose (PnP-based estimation)
Side Camera (Raspberry Pi Camera v2.1)	• Purpose: Fine alignment & trigger detection • Methods: - Hough Circle detection (receptacle) - HSV thresholding (Station B LED trigger)

2.4 Payload Delivery Mechanism

Launcher System	• Type: Cam-based spring-loaded mechanism • Actuation: UART-controlled servo motor (DC mode) • Function: Converts rotational motion into impulse for ball launching
Feeding Mechanism	• Type: Gravity-fed tube • Capacity: 9 ping pong balls • Function: Passive sequential reloading



3. Technical Guide

Setup

1. Turn on the robot switch.
2. Load 9 ping pong balls into the feeder.
3. Place the TurtleBot3 at the start line.

Launch

Run the command on the laptop terminal:

```
start
```

The robot begins moving autonomously.

Phase 1: Exploration

The robot navigates the maze autonomously using frontier exploration, systematically visiting unexplored regions while avoiding obstacles. As it moves, Cartographer builds a 2D occupancy map via SLAM, which can be monitored in real time through Foxglove or RViz.

Exploration continues until a station is detected.

Phase 2: Docking

When the robot's front camera detects an ArUco marker above a station receptacle, it interrupts exploration and begins the docking approach. The robot advances toward the marker in short incremental steps, pausing between each step to re-read the marker's pose and correct its bearing. It aims to arrive approximately 30cm from the marker, aligned as perpendicular to the wall as possible.

Once within docking range, the robot rotates so that its side-mounted launcher faces the receptacle. The side camera then takes over, detecting the circular opening of the receptacle using Hough circle detection. The robot adjusts forward or backwards until the receptacle opening is centred in the camera frame.

Phase 3: Firing

The firing sequence depends on which station was detected.

Station A (static receptacle): The receptacle is fixed in place. Once aligned, the launcher immediately fires a ball, waits 7 seconds, then fires a second ball and waits 3 seconds, then fires the third ball.

Station B (moving receptacle): The receptacle has a tin can that travels back and forth on a motorised rail. A blue LED inside the tin can signals its presence behind the cutout opening. After aligning to the wooden receptacle frame, the robot watches for blue light through the opening using the side camera. When the blue LED is detected and confirmed over several consecutive frames, the launcher fires. The robot then waits for the tin can to return on its next pass before firing the second and third balls.

If a station is left, the robot will continue exploring to find the next station. If both stations are complete, the mission ends.

4. Acceptable Defect Log

Part	Acceptable Defect
Cam and shaft	• Shaft not able to be fully hammered into cam
Feeder pipe	• Pipe blocker fastened using superglue • One feeder mount is not in full contact with the waffle plate • Edges are rough
LiDAR	• LiDAR supports are slightly bent
Launcher housing	• Launcher mount has warped bottom due to excess heat during 3D printing • There is some tape inside launcher
Dowels	• Edges of dowels are rough
Rpi Camera Mount	• Rpi camera edges are not fully within the mount's edges
Selected nuts and bolts	• Launcher bolts have no nuts due to inaccessible locations. Instead they are screwed in • Some nuts are fully tightened but are not seated flush against bolt due to waffle plate thickness
All electronic components	• Open and vulnerable to wear and tear • Unorganised wires

5. Factory Acceptance Test

CDE2310 AMR FAT		
Mechanical	Task	Check
	OpenCR riveted down in the correct orientation	✓
	Raspberry Pi bolted down	✓
	Front and side cameras are mounted and active	✓
	Shaft and Cam are mated tightly	✓
	No loose screws, nuts or bolts present	✓
	Servo is mounted	✓
	Feeder is oiled, mounted and not blocking LiDAR	✓
Electrical	The battery is fully charged, working and plugged into OpenCR	✓
	OpenCR boots up normally and sings a tune	✓
	Servo control board, USB Camera, LiDAR, and OpenCR plugged into RPi USB ports and working	✓
	Servo control board pins powered by RPi power pins	✓
	OpenCR power and ground pins connected to RPi power and ground pins	✓
	Wires and cables are undamaged and unexposed	✓

Software	Laptop can SSH into Turtlebot RPi	✓
	Laptop and RPi are on the same ROS Discovery Network	✓
	Cartographer, Nav2, Frontier Exploration Nodes are working	✓
	Foxglove Visualiser is working	✓
	Rosbu is activated	✓

6. Maintenance and Part Replacement Log

Components	Description
Raspberry Pi 4B 1	Plugged 5V pin of OpenCR into GND pin of RPi and GND pin of OpenCR into 5V pin of RPi, damaged the RPi with reverse voltage. Replaced
Raspberry Pi 4B 2	Dropped nut on top of live RPi while unscrewing a bolt, possibly causing the RPi to short-circuit, resulting in a permanent undervoltage characterised by a persistent red light with regular flashing green lights. Furthermore, running computationally intensive tasks on the RPi like Hough circle detection, might have caused the RPi to overheat and accumulate damage. Replaced.
OpenCR 1.0	Possibly the same reason as RPi, due to doing mechanical work on live electronics, the OpenCR status light is not working, and is unable to connect to RPi. Replaced.
Servo Wires	Servo jumper wires keep disconnecting from RPi power pins, replaced with different wires
Launcher, Pipe Feeder and Camera Mount	Positions of the pipe, Launcher mechanism and camera mount keep adjusting in order to accommodate new requirements to launcher angle, gravity feeding and camera alignment
Cam and Shaft	The cam and shaft are repeatedly reprinted and assembled due to servo's inability to rotate clockwise (which unscrews the servo disc), strength requirements, tight fit between cam and shaft and more. Prototyped cam and shaft through multiple iterations. Reprogrammed the servo to only turn clockwise when launching and initialising, and hammered the shaft and cam together tightly to ensure a friction fit.